The Mathematics of `demo.py`

A 7-microphone superdirective beamformer, streamed through an overlap-add STFT

November 13, 2026

Abstract

`demo.py` turns the seven MEMS microphones of a miniDSP UMA-8 into one "acoustic zoom lens" pointed at the front of the board, and plays the result to headphones in real time. This document derives every equation the program uses, from the plane-wave model through the superdirective weight design to the short-time Fourier transform (STFT) that streams it. Each section works the same way: *intuition* first, then a *derivation*, then a *code map* with the exact lines of `demo.py`, then the *design rationale* behind the parameter choices. All plotted numbers were computed from the code itself (`design_weights`, `steering_vectors`), so the figures match what `python demo.py -pattern` prints.

Contents

1 Overview **1**

2 Array geometry **2**

3 Plane waves and the steering vector **4**

4 Distortionless beamforming and delay-and-sum **6**

5 Superdirective (MVDR) beamforming against diffuse noise **7**
5.1 The diffuse noise field . . . 7
5.2 The optimal weights . . . 8
5.3 Why the diagonal loading is essential . . . 8

6 Beam pattern, white-noise gain and spatial aliasing **9**
6.1 Spatial aliasing and the front–back lobe . . . 10

7 Streaming: STFT analysis, beamforming, overlap-add synthesis **12**
7.1 What one call to `process` computes . . . 13
7.2 The window and perfect reconstruction . . . 13
7.3 Circular convolution and how much it matters . . . 13

8 Gain staging, output and level metering **15**

A Parameters and their effects **16**

B Symbols **16**

1 Overview

Sound from a talker in front of the array reaches the seven microphones at slightly different times. The pattern of delays depends only on the direction the sound comes from. In the frequency domain each delay becomes a phase rotation. So at every frequency, "listening in a direction" means choosing seven complex weights that make the phasors from the look direction add up exactly, while phasors from other directions partly cancel. Figure 1 shows the complete signal chain.

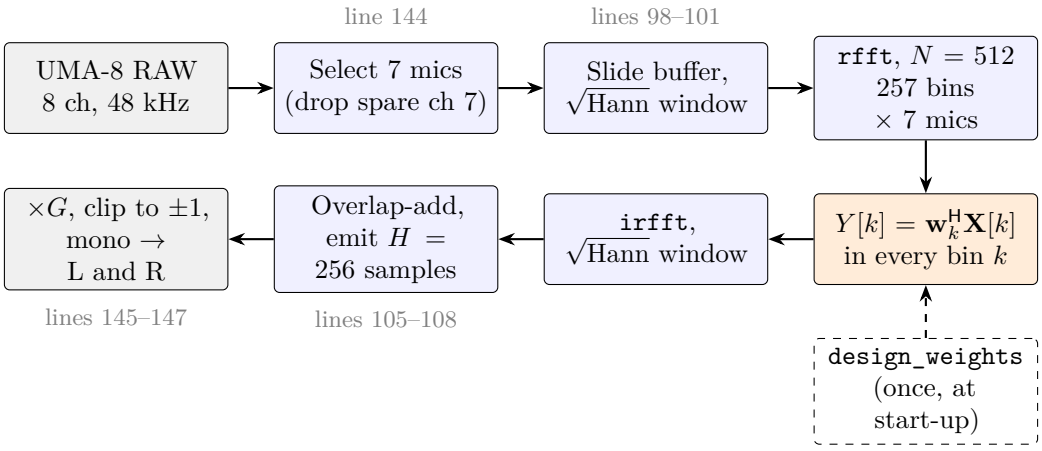


Figure 1: The signal chain in `demo.py`. Every audio callback receives $H = 256$ new samples per channel and returns H output samples. The weights are computed once.

Notation. $M = 7$ microphones; sample rate $f_s = 48\,000$ Hz; hop $H = 256$; FFT length $N = 2H = 512$; speed of sound $c = 343$ m/s; ring radius $r = 0.043$ m. Bold lower-case letters are column vectors in $\mathbb{C}^M$ (one entry per microphone). $(\cdot)^{\mathsf{T}}$ is the transpose and $(\cdot)^{\mathsf{H}}$ the conjugate transpose. Frequency bin $k \in \{0, \ldots, N/2\}$ has centre frequency $f_k = kf_s/N = k \cdot 93.75$ Hz. The acoustic wavenumber is $\kappa = 2\pi f/c$ (rad/m); the letter k is reserved for the bin index. Table 1 maps each part of the code to a section of this document.

Code object	Lines	What it computes	Section
constants	6–35	geometry, physics, gains	2, App. A
`mic_layout`	48–55	mic positions $\mathbf{p}_m$	2
`steering_vectors`	58–63	$\mathbf{d}(f, \phi)$	3
`design_weights` (`das`)	72–73	$\mathbf{w} = \mathbf{d}/M$	4
`design_weights` (superdirective)	75–82	$\mathbf{w} = \mathbf{\Gamma}_\mu^{-1}\mathbf{d}/(\mathbf{d}^{\mathsf{H}}\mathbf{\Gamma}_\mu^{-1}\mathbf{d})$	5
`print_beampattern`	112–131	$B(f, \phi)$, WNG	6
`StreamingBeamformer`	85–109	STFT, overlap-add	7
`audio_callback`, `main`	135–147, 196–201	gain, clip, routing	8
`show_levels`	150–167	per-channel dBFS	8

Table 1: Where each piece of `demo.py` is explained.

Design rationale: why work in the frequency domain?

The delays involved are tiny. The largest is $r/c \approx 125\,\mu$s, about 6 samples, and most are a fraction of a sample. A fractional delay is awkward in the time domain (it needs an interpolation filter) but exact in the frequency domain: it is just a phase factor. The superdirective weights are also *frequency-dependent by design* (Section 5), so a per-bin design is the natural form. The STFT gives both, at the cost of one N-point FFT per channel per hop.

2 Array geometry

```
# --- UMA-8 array geometry ------------------------------------------------
# 1 centre mic + 6 mics on a 43 mm ring (hexagon), looking down from the top.
# Front = away from the USB cable = 12 o'clock = the beam direction.
# Axes: x = front, y = left. Angles are degrees from front, positive to the left.
# Mic 1 is directly right; going clockwise in 60-degree steps gives mics 2-6,
# so the beam sits halfway between mic 5 (30 deg left) and mic 6 (30 deg right).
# Mic N is assumed to arrive on USB channel N; verify with --levels.
CENTER_CHANNEL = 0                    # mic 0, centre of the array
RING_MIC_ANGLES_DEG = {               # USB channel: angle of that mic
    1: -90.0,                         # 3 o'clock, directly right
    2: -150.0,                        # 5 o'clock, back right
    3: 150.0,                         # 7 o'clock, back left
    4: 90.0,                          # 9 o'clock, directly left
    5: 30.0,                          # 11 o'clock, front left
    6: -30.0,                         # 1 o'clock, front right
}
SPARE_CHANNEL = 7                     # spare PDM input, ignored
RING_RADIUS = 0.043                   # metres
```

The UMA-8 has one centre microphone and six on a hexagon of radius $r = 43\,\text{mm}$ (Figure 2). The axes are x = front (away from the USB cable) and y = left, and azimuth ϕ is measured from the front, positive to the left. Mic m on the ring at angle θ_m sits at

$$\mathbf{p}_m = r \begin{bmatrix} \cos\theta_m \\ \sin\theta_m \end{bmatrix}, \qquad \mathbf{p}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \tag{1}$$

```
def mic_layout():
    """Return (channel indices, mic xy positions in metres) for the 7 mics."""
    ring_channels = list(RING_MIC_ANGLES_DEG)
    angles = np.radians(list(RING_MIC_ANGLES_DEG.values()))
    ring = RING_RADIUS * np.stack([np.cos(angles), np.sin(angles)], axis=1)
    positions = np.vstack([[0.0, 0.0], ring])
    channels = [CENTER_CHANNEL] + ring_channels
    return channels, positions
```

Line 52 evaluates (1) for all six ring mics at once. `np.stack(..., axis=1)` builds a 6×2 matrix with one row per mic. Line 53 prepends the centre mic, so `positions` is 7×2, and row m belongs to USB channel `channels[m]`.

Proposition 2.1 (Inter-mic distances)**.** *The 21 microphone pairs have only three distinct distances:* $r = 43\,mm$, $\sqrt{3}\,r \approx 74.5\,mm$ *and* $2r = 86\,mm$.

Proof. The centre mic is at distance r from each ring mic. Two ring mics separated by angle $\Delta\theta$ form a chord of length $2r\sin(\Delta\theta/2)$. On a hexagon $\Delta\theta \in \{60°, 120°, 180°\}$, which gives r, $\sqrt{3}\,r$ and $2r$. □

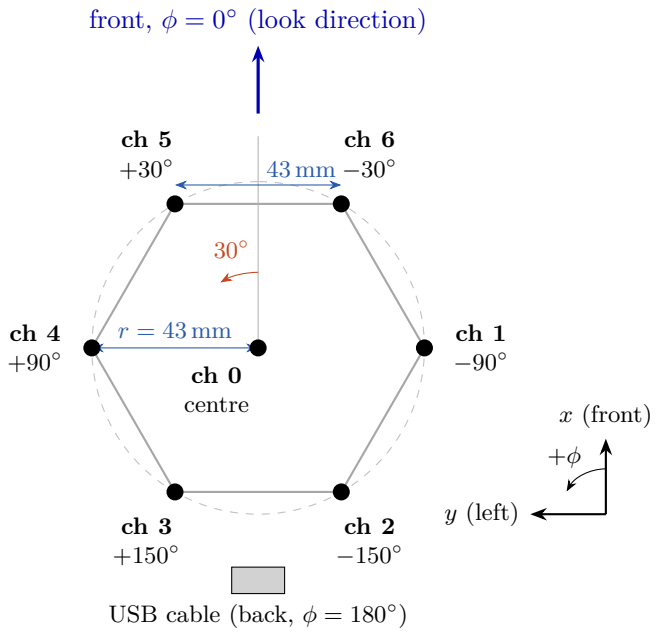


Figure 2: UMA-8 seen from above, with channel numbers as assumed on lines 20–27. The beam points between ch 5 and ch 6. The geometry is mirror-symmetric about the x-axis, with pairs (5,6), (4,1) and (3,2).

These three numbers set every frequency scale that follows. The smallest spacing sets the spatial-aliasing limit (Section 6), and the aperture $D = 2r$ determines how directional the array can be at low frequencies.

Proposition 2.2 (Left/right symmetry)**.** *With the look direction at $\phi_0 = 0$, both designs satisfy $|B(f,\phi)| = |B(f,-\phi)|$. The beam pattern is a mirror image about the front axis, so plotting $0° \le \phi \le 180°$ shows all of it.*

Proof. Reflection $y \mapsto -y$ maps the set of mic positions onto itself. It swaps channels $5 \leftrightarrow 6$, $4 \leftrightarrow 1$, $3 \leftrightarrow 2$ and fixes 0. Write $\mathbf{P}$ for the corresponding symmetric permutation matrix ($\mathbf{P} = \mathbf{P}^{\mathsf{T}} = \mathbf{P}^{-1}$). From (4) below, $\mathbf{d}(f,-\phi) = \mathbf{P}\mathbf{d}(f,\phi)$, and in particular $\mathbf{P}\mathbf{d}(f,0) = \mathbf{d}(f,0)$. Distances are invariant under the reflection, so $\mathbf{P}\boldsymbol{\Gamma}\mathbf{P} = \boldsymbol{\Gamma}$. The weights in Sections 4 and 5 are the *unique* solutions of problems that are invariant under $\mathbf{P}$, so $\mathbf{P}\mathbf{w} = \mathbf{w}$. Therefore $B(f,-\phi) = \mathbf{w}^{\mathsf{H}}\mathbf{P}\mathbf{d}(f,\phi) = (\mathbf{P}\mathbf{w})^{\mathsf{H}}\mathbf{d}(f,\phi) = B(f,\phi)$. □

Remark 2.3 (Up/down ambiguity). The array is flat, so a source at elevation $+\epsilon$ and its mirror image at $-\epsilon$ produce identical steering vectors. In 3-D, the "beam" is really a fan that is symmetric about the plane of the board. Only directions *within* the plane are modelled (Section 3).

Design rationale

Including the centre mic adds a seventh observation at the phase reference. It raises the best-case white-noise gain from $10\log_{10}6 = 7.8\,\text{dB}$ to $10\log_{10}7 = 8.5\,\text{dB}$ (Proposition 4.2) and gives the superdirective solver one more degree of freedom. **Dropping the spare channel** (`SPARE_CHANNEL = 7`) happens implicitly on line 144, where `indata[:, beamformer.channels]` keeps only the columns $[0, 1, \ldots, 6]$. **The channel mapping is an assumption.** Run `-levels` and tap each mic to confirm it (Section 8). A wrong mapping does

not crash anything; it silently steers the beam somewhere else.

3 Plane waves and the steering vector

Intuition

A distant talker produces wavefronts that are effectively flat by the time they cross a 9 cm board. A mic that is further "upstream" (closer to the talker) hears the sound *earlier*. How much earlier is the distance it sits upstream, divided by c. That distance is just the projection of the mic's position onto the direction of the source (Figure 3).

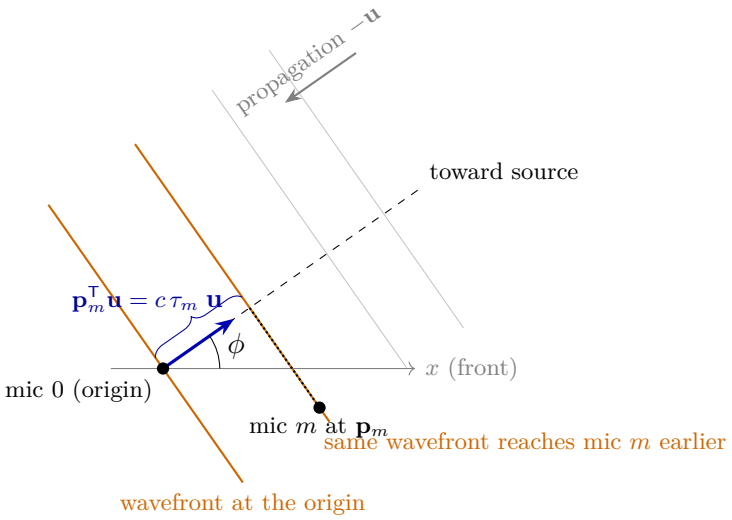


Figure 3: A plane wave from azimuth ϕ. Mic m sits $\mathbf{p}_m^\mathsf{T}\mathbf{u}$ metres upstream of the origin, so it hears the wave $\tau_m = \mathbf{p}_m^\mathsf{T}\mathbf{u}/c$ seconds earlier (lines 61–62).

Derivation. Let $\mathbf{u} = [\cos\phi,\ \sin\phi]^\mathsf{T}$ be the unit vector pointing *toward* the source (line 61). A plane wave that carries the signal $s(t)$ and travels in direction $-\mathbf{u}$ has pressure field

$$p(\mathbf{x}, t) = s\left(t + \frac{\mathbf{u}^\mathsf{T}\mathbf{x}}{c}\right). \tag{2}$$

This is the right sign: points with $\mathbf{u}^\mathsf{T}\mathbf{x} > 0$ are closer to the source, so they see a given part of s earlier. Mic m therefore records $x_m(t) = s(t + \tau_m)$, with

$$\tau_m = \frac{\mathbf{p}_m^\mathsf{T}\mathbf{u}}{c} \qquad \text{(line 62, } \texttt{advance = positions @ direction / SPEED_OF_SOUND}\text{)}. \tag{3}$$

NumPy's FFT uses the kernel $e^{-j2\pi ft}$. With that convention the shift theorem reads $s(t+\tau) \longleftrightarrow e^{+j2\pi f\tau}S(f)$. Stacking the seven mics gives

$$\mathbf{X}(f) = S(f)\,\mathbf{d}(f,\phi), \qquad \mathbf{d}(f,\phi) = \begin{bmatrix} e^{j2\pi f\tau_0} \\ \vdots \\ e^{j2\pi f\tau_6} \end{bmatrix} = \exp\left(j\kappa\,\mathbf{p}_m^\mathsf{T}\mathbf{u}\right)_{m=0}^{6}. \tag{4}$$

The vector $\mathbf{d}$ is the *steering vector*: the array's fingerprint for direction ϕ at frequency f. All of its entries have modulus one, so $\|\mathbf{d}\|^2 = M$. Because $\mathbf{p}_0 = \mathbf{0}$, $d_0 \equiv 1$: the centre mic is the phase reference.

```
58 def steering_vectors(positions, azimuth_deg, freqs):
59     """Phase of a plane wave from azimuth_deg at each mic, shape (F, M)."""
60     az = np.radians(azimuth_deg)
61     direction = np.array([np.cos(az), np.sin(az)])
62     advance = positions @ direction / SPEED_OF_SOUND
63     return np.exp(2j * np.pi * freqs[:, None] * advance[None, :])
```

Code map. Line 63 evaluates (4) for every bin at once. `freqs[:, None] * advance[None, :]` is the outer product $f_k \tau_m$, which gives an $F \times M$ array with $F = N/2{+}1 = 257$ (`np.fft.rfftfreq(512, 1/48000)`, line 68, returns $0, 93.75, \ldots, 24\,000$ Hz).

Assumptions, and when they hold.

1. *Far field.* The wavefronts are flat if the source distance R satisfies the Fraunhofer criterion $R > 2D^2/\lambda$. With $D = 86$ mm this is 0.17 m at 4 kHz and even less at lower frequencies, so any talker further away than about an arm's length is in the far field.

2. *In-plane, free-field propagation.* Elevation, scattering off the board and the mics' own responses are ignored. All mics are assumed identical and omnidirectional.

3. *Multiplicative STFT model.* Equation (4) is exact for the continuous Fourier transform. For an N-sample STFT frame it holds approximately as long as $|\tau_m| \ll N/f_s$. Here $\max|\tau_m| = 125\,\mu$s ≈ 6 samples, compared with a frame of 512 samples.

4 Distortionless beamforming and delay-and-sum

A (narrowband) beamformer forms one output per bin as a linear combination of the mics:

$$Y(f) = \mathbf{w}(f)^{\mathsf{H}}\mathbf{X}(f) = \sum_{m=0}^{6} \overline{w_m(f)}\, X_m(f). \tag{5}$$

The conjugate is a convention that makes Y an inner product. The code follows it exactly: `w_h = weights.conj()` (line 92) and `w[k].conj() @ d` (line 127).

Signal model and performance measures. Write each STFT bin as target plus noise, $\mathbf{X} = S\mathbf{d} + \mathbf{V}$, with noise covariance $\mathbb{E}[\mathbf{V}\mathbf{V}^{\mathsf{H}}] = \sigma_v^2\mathbf{\Gamma}$, where $\Gamma_{mm} = 1$ (so $\mathbf{\Gamma}$ is a *coherence* matrix). Then

$$Y = S\,(\mathbf{w}^{\mathsf{H}}\mathbf{d}) + \mathbf{w}^{\mathsf{H}}\mathbf{V}, \qquad \mathbb{E}|\mathbf{w}^{\mathsf{H}}\mathbf{V}|^2 = \sigma_v^2\,\mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}\mathbf{w}. \tag{6}$$

Definition 4.1 (Distortionless constraint; array gain)**.** A beamformer is *distortionless* toward ϕ_0 if $\mathbf{w}^{\mathsf{H}}\mathbf{d}(f, \phi_0) = 1$ at every f. Its *array gain* against noise field $\mathbf{\Gamma}$ is the output SNR divided by the SNR at a single mic:

$$A_{\mathbf{\Gamma}}(\mathbf{w}) = \frac{|\mathbf{w}^{\mathsf{H}}\mathbf{d}|^2}{\mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}\mathbf{w}}. \tag{7}$$

Two cases matter. The *white-noise gain* is $\mathrm{WNG} = A_{\mathbf{I}}$, for mic self-noise, which is independent from mic to mic. The *directivity index* is $\mathrm{DI} = A_{\mathbf{\Gamma}_{\mathrm{diff}}}$, for reverberation and babble arriving from all directions (Section 5).

The distortionless constraint is why the talker's voice comes out with its original timbre. Whatever happens to the noise, the target spectrum is multiplied by exactly 1 in every bin.

```
72      if method == "das":                          # delay-and-sum
73          return d / m
```

Proposition 4.2 (Delay-and-sum is the WNG-optimal distortionless beamformer)**.** *Among all* $\mathbf{w}$ *with* $\mathbf{w}^{\mathsf{H}}\mathbf{d} = 1$*, the white-noise gain* $1/\|\mathbf{w}\|^2$ *is maximised uniquely by* $\mathbf{w} = \mathbf{d}/M$ *(line 73), and the maximum is* $\mathrm{WNG} = M$*, i.e.* $10\log_{10} 7 = 8.45$ *dB.*

Proof. By Cauchy–Schwarz, $1 = |\mathbf{w}^{\mathsf{H}}\mathbf{d}|^2 \le \|\mathbf{w}\|^2\|\mathbf{d}\|^2 = M\|\mathbf{w}\|^2$, so $\mathrm{WNG} = 1/\|\mathbf{w}\|^2 \le M$. Equality holds iff $\mathbf{w} \propto \mathbf{d}$, and the constraint fixes the constant: $\mathbf{w}^{\mathsf{H}}\mathbf{d} = \mathbf{d}^{\mathsf{H}}\mathbf{d}/M = 1$. □

Intuition

$\overline{d_m} = e^{-j2\pi f\tau_m}$ undoes each mic's head start, so the seven copies of the target line up and are averaged. Independent self-noise does not line up, so its power drops by a factor of M.

Why delay-and-sum is not enough here. As $f \to 0$, every steering vector tends to $\mathbf{1}$, whatever the direction. Then $B(f,\phi) = \mathbf{d}(f,0)^{\mathsf{H}}\mathbf{d}(f,\phi)/M \to 1$ for all ϕ, and the array is omnidirectional. Quantitatively, at 281 Hz the wavelength is 1.22 m, fourteen times the aperture. DAS's directivity index there is only 0.15 dB (Figure 6), while most of the energy in speech and room noise lies below 1 kHz. Getting directivity out of a small array needs a different criterion.

5 Superdirective (MVDR) beamforming against diffuse noise

5.1 The diffuse noise field

Reverberation and distant babble behave like a *spherically isotropic* (diffuse) field: infinitely many mutually uncorrelated plane waves of equal power, arriving uniformly from all directions in 3-D.

Proposition 5.1 (Diffuse-field coherence)**.** *In a diffuse field, the coherence between two omnidirectional mics a distance* d_{ij} *apart is*

$$\Gamma_{ij}(f) = \frac{\sin(\kappa d_{ij})}{\kappa d_{ij}}, \qquad \kappa = \frac{2\pi f}{c}. \tag{8}$$

Proof. Each plane wave from direction $\mathbf{u} \in S^2$ contributes $e^{j\kappa\mathbf{u}^{\mathsf{T}}\mathbf{p}_i}$ to mic i. The waves are uncorrelated and have equal power, so

$$\Gamma_{ij} = \frac{1}{4\pi}\oint_{S^2} e^{j\kappa\,\mathbf{u}^{\mathsf{T}}(\mathbf{p}_i-\mathbf{p}_j)}\,d\Omega(\mathbf{u}).$$

Choose the polar axis along $\mathbf{p}_i - \mathbf{p}_j$, so that $\mathbf{u}^{\mathsf{T}}(\mathbf{p}_i - \mathbf{p}_j) = d_{ij}\cos\vartheta$. The azimuthal integral contributes 2π. Substituting $\mu = \cos\vartheta$,

$$\Gamma_{ij} = \frac{1}{2}\int_0^{\pi} e^{j\kappa d_{ij}\cos\vartheta}\sin\vartheta\,d\vartheta = \frac{1}{2}\int_{-1}^{1} e^{j\kappa d_{ij}\mu}\,d\mu = \frac{\sin(\kappa d_{ij})}{\kappa d_{ij}}.$$ □

NumPy's `np.sinc(x)` is the *normalised* sinc, $\sin(\pi x)/(\pi x)$. With $x = 2fd/c$ we get $\pi x = 2\pi fd/c = \kappa d$, so line 79 is exactly (8). `dist` (line 76) holds the 7×7 matrix of d_{ij}, and its diagonal gives $\Gamma_{ii} = \mathrm{sinc}(0) = 1$.

Remark 5.2. The array is planar, but room noise is three-dimensional, so the 3-D (spherical) average is the right model. A purely 2-D (cylindrical) field would give $J_0(\kappa d)$ instead.

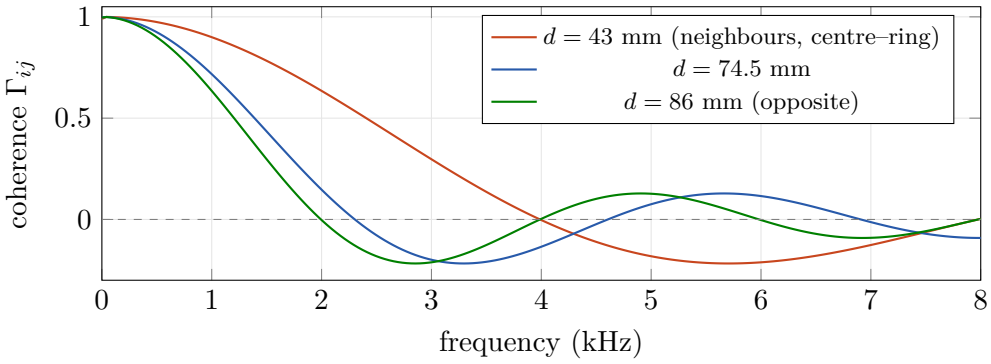


Figure 4: Diffuse-noise coherence (8) for the three spacings of Proposition 2.1. Below about 1 kHz every pair is almost perfectly coherent.

Intuition

Below about 1 kHz, every pair of mics hears nearly the *same* diffuse noise (Figure 4). Noise that is common to all mics can be removed by *subtracting* them, much as a differential (gradient) microphone does. A sound from the front is not identical at all mics, because it carries the small phase slope $\mathbf{d}$. So a carefully chosen set of differences can cancel most of the diffuse field and still pass the target. The price is that differencing nearly identical signals amplifies anything that is *not* common, in particular each mic's own self-noise. That trade-off is the whole story of diagonal loading below.

5.2 The optimal weights

Proposition 5.3 (MVDR / superdirective solution)**.** *Let* $\mathbf{\Gamma}_\mu = \mathbf{\Gamma} + \mu\mathbf{I}$ *with* $\mu > 0$*. The problem*

$$\min_{\mathbf{w}} \ \mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}_\mu\mathbf{w} \quad \textit{subject to} \quad \mathbf{w}^{\mathsf{H}}\mathbf{d} = 1 \tag{9}$$

has the unique solution

$$\boxed{\mathbf{w} = \frac{\mathbf{\Gamma}_\mu^{-1}\mathbf{d}}{\mathbf{d}^{\mathsf{H}}\mathbf{\Gamma}_\mu^{-1}\mathbf{d}}} \qquad \textit{with minimum value} \qquad \frac{1}{\mathbf{d}^{\mathsf{H}}\mathbf{\Gamma}_\mu^{-1}\mathbf{d}}. \tag{10}$$

Proof. $\mathbf{\Gamma}$ is an average of rank-one matrices $\mathbf{a}\mathbf{a}^{\mathsf{H}}$ (see the proof of Proposition 5.1), so it is Hermitian positive semidefinite. Hence $\mathbf{\Gamma}_\mu$ is positive definite and has a positive-definite square root. Apply Cauchy–Schwarz to the vectors $\mathbf{\Gamma}_\mu^{1/2}\mathbf{w}$ and $\mathbf{\Gamma}_\mu^{-1/2}\mathbf{d}$:

$$1 = |\mathbf{w}^{\mathsf{H}}\mathbf{d}|^2 = \left|(\mathbf{\Gamma}_\mu^{1/2}\mathbf{w})^{\mathsf{H}}(\mathbf{\Gamma}_\mu^{-1/2}\mathbf{d})\right|^2 \le (\mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}_\mu\mathbf{w})\,(\mathbf{d}^{\mathsf{H}}\mathbf{\Gamma}_\mu^{-1}\mathbf{d}).$$

So $\mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}_\mu\mathbf{w} \ge 1/(\mathbf{d}^{\mathsf{H}}\mathbf{\Gamma}_\mu^{-1}\mathbf{d})$, with equality iff $\mathbf{\Gamma}_\mu^{1/2}\mathbf{w} \propto \mathbf{\Gamma}_\mu^{-1/2}\mathbf{d}$, i.e. $\mathbf{w} \propto \mathbf{\Gamma}_\mu^{-1}\mathbf{d}$. The constraint fixes the scale and gives (10). □

This is the same argument as Proposition 4.2, carried out in the inner product weighted by $\mathbf{\Gamma}_\mu$. Delay-and-sum is simply the special case $\mathbf{\Gamma}_\mu = \mathbf{I}$.

```
    # Superdirective (MVDR against diffuse noise): narrower beam at low frequencies
    dist = np.linalg.norm(positions[:, None] - positions[None, :], axis=-1)
```

```
    w = np.empty_like(d)
    for k, f in enumerate(freqs):
        gamma = np.sinc(2 * f * dist / SPEED_OF_SOUND) + DIAGONAL_LOADING * np.eye(m)
        g_inv_d = np.linalg.solve(gamma, d[k])
        w[k] = g_inv_d / (d[k].conj() @ g_inv_d)
    return w
```

Code map. Line 79 builds $\mathbf{\Gamma}_\mu(f_k)$. Line 80 computes $\mathbf{\Gamma}_\mu^{-1}\mathbf{d}$ by solving the linear system rather than forming an inverse, which is cheaper and numerically more accurate. Line 81 divides by $\mathbf{d}^{\mathsf{H}}\mathbf{\Gamma}_\mu^{-1}\mathbf{d}$, a real positive scalar because $\mathbf{\Gamma}_\mu^{-1}$ is Hermitian positive definite. The loop runs 257 small 7×7 solves, *once*, at start-up (line 197). The real-time callback never touches this code.

5.3 Why the diagonal loading is essential

Ill-conditioning at low frequency. As $f \to 0$, (8) gives $\mathbf{\Gamma} \to \mathbf{1}\mathbf{1}^{\mathsf{T}}$, a rank-one matrix, so $\mathbf{\Gamma}^{-1}$ blows up. The condition numbers of the actual 7×7 matrices (computed from the code's geometry) are:

f	100 Hz	300 Hz	1 kHz	3 kHz
$\mathrm{cond}(\mathbf{\Gamma})$	1.2×10^{10}	1.7×10^{7}	1.1×10^{4}	7.8
$\mathrm{cond}(\mathbf{\Gamma} + 0.05\,\mathbf{I})$	141	139	117	6.6

Without loading, the unconstrained weights at 100 Hz would be huge and of alternating sign. They would cancel the diffuse field perfectly *on paper*, but they would amplify mic self-noise by the factor $\|\mathbf{w}\|^2 = 1/\mathrm{WNG}$, which is enormous.

Three equivalent readings of μ.

1. *Tikhonov regularisation.* $\mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}_\mu\mathbf{w} = \mathbf{w}^{\mathsf{H}}\mathbf{\Gamma}\mathbf{w} + \mu\|\mathbf{w}\|^2$. The objective adds a penalty on the weight norm, i.e. on self-noise amplification, to the diffuse-noise power.

2. *A physical noise model.* $\mathbf{\Gamma}_\mu$ is the coherence of diffuse noise plus uncorrelated sensor noise whose power is μ times the diffuse power. The design is optimal for that mixed field.

3. *A dial between two designs.* At $\mu \to 0$ the design is fully superdirective. As $\mu \to \infty$ it becomes delay-and-sum. To see the second limit, expand $\mathbf{\Gamma}_\mu^{-1} = \mu^{-1}(\mathbf{I} + \mathbf{\Gamma}/\mu)^{-1} = \mu^{-1}(\mathbf{I} - \mathbf{\Gamma}/\mu + O(\mu^{-2}))$. Then $\mathbf{\Gamma}_\mu^{-1}\mathbf{d} \propto \mathbf{d} + O(\mu^{-1})$, and after normalising, $\mathbf{w} \to \mathbf{d}/M$.

The measured trade-off. The table sweeps μ at the 281 Hz bin, using the code's geometry. WNG and DI are defined in (7), and "back" is $|B|$ at $\phi = 180°$.

μ	0.001	0.01	**0.05 (used)**	0.2	1	10
WNG (dB)	−13.6	−5.8	**−4.0**	−0.2	+6.2	+8.4
DI (dB)	6.8	6.1	**5.5**	3.6	1.2	0.3
back (dB)	−9.1	−7.7	**−14.2**	−12.2	−2.8	−0.6
$\max_m \lvert w_m \rvert$	2.42	0.94	**0.78**	0.50	0.22	0.14

Design rationale: `DIAGONAL_LOADING = 0.05`

At $\mu = 0.05$ the 281 Hz beam gains 5.4 dB of directivity over delay-and-sum (5.5 vs 0.15 dB). In exchange, mic self-noise is 4 dB *louder* than with a single mic, and 12.4 dB louder than with DAS. Going down to $\mu = 0.001$ buys only 1.3 dB more DI and costs almost 10 dB more hiss. Going up to $\mu = 1$ throws most of the directivity away. So 0.05 sits near the knee of the curve. The deep rear null at 0.05 is a helpful side effect of where the null lands, not something the criterion optimises. If the output hisses in a quiet room, raise μ. If it sounds roomy, lower it.

Limiting cases that check the code.

- *DC bin* ($f = 0$): $\mathbf{d} = \mathbf{1}$ and $\mathbf{\Gamma}_\mu = \mathbf{1}\mathbf{1}^\mathsf{T} + \mu\mathbf{I}$. Since $\mathbf{\Gamma}_\mu\mathbf{1} = (M + \mu)\mathbf{1}$, we get $\mathbf{\Gamma}_\mu^{-1}\mathbf{1} = \mathbf{1}/(M + \mu)$ and therefore $\mathbf{w} = \mathbf{1}/M$: exactly delay-and-sum.
- *High frequency*: the off-diagonal entries of $\mathbf{\Gamma}$ decay like $1/(\kappa d)$, so $\mathbf{\Gamma} \to \mathbf{I}$ and the superdirective weights tend to DAS. This is visible in Figure 6: the curves merge above roughly 3.5 kHz, where $\mathrm{cond}(\mathbf{\Gamma})$ is already below 8.

6 Beam pattern, white-noise gain and spatial aliasing

Definition 6.1 (Beam pattern)**.** For weights designed toward ϕ_0, the response to a unit plane wave from ϕ is

$$B(f, \phi) = \mathbf{w}(f)^\mathsf{H}\mathbf{d}(f, \phi), \qquad B_{\mathrm{dB}} = 20\log_{10}|B|. \tag{11}$$

The distortionless constraint gives $B(f, \phi_0) = 1 = 0\,\mathrm{dB}$.

```
112 def print_beampattern(method):
113     """Print the beam's response (dB) vs azimuth so you can see the cone width."""
114     _, positions = mic_layout()
115     n_fft = 2 * BLOCK_SIZE
116     w = design_weights(positions, LOOK_AZIMUTH_DEG, method, n_fft)
117     freqs = np.fft.rfftfreq(n_fft, 1 / SAMPLE_RATE)
118     azimuths = np.arange(0, 181, 15)
119 
120     print(f"{method}: response in dB vs azimuth (degrees from front)\n")
121     print("  freq  " + "".join(f"{a:>6}" for a in azimuths) + "   white-noise gain")
122     for f in (250, 500, 1000, 2000, 4000, 8000):
123         k = int(np.argmin(np.abs(freqs - f)))
124         row = []
125         for az in azimuths:
126             d = steering_vectors(positions, LOOK_AZIMUTH_DEG + az, freqs[k:k + 1])[0]
127             row.append(20 * np.log10(abs(w[k].conj() @ d) + 1e-12))
128         wng = 10 * np.log10(1 / np.sum(abs(w[k]) ** 2))
129         print(f"{freqs[k]:6.0f}  " + "".join(f"{r:6.1f}" for r in row) + f"   {wng:6.1f} dB")
130     print("\nWhite-noise gain below 0 dB means mic self-noise is boosted;"
131           "\nraise DIAGONAL_LOADING if the output hisses.")
```

Code map. Line 126 builds $\mathbf{d}(f_k, \phi_0 + \phi)$ for each test azimuth. Line 127 evaluates $20\log_{10}|\mathbf{w}^\mathsf{H}\mathbf{d}|$; the `1e-12` keeps $\log 0$ away at exact nulls. Line 128 is WNG $= 1/\|\mathbf{w}\|^2$, which is valid because $|\mathbf{w}^\mathsf{H}\mathbf{d}| = 1$. Line 123 rounds each requested frequency to the nearest bin, which is why the printout shows 281 Hz rather than 250.

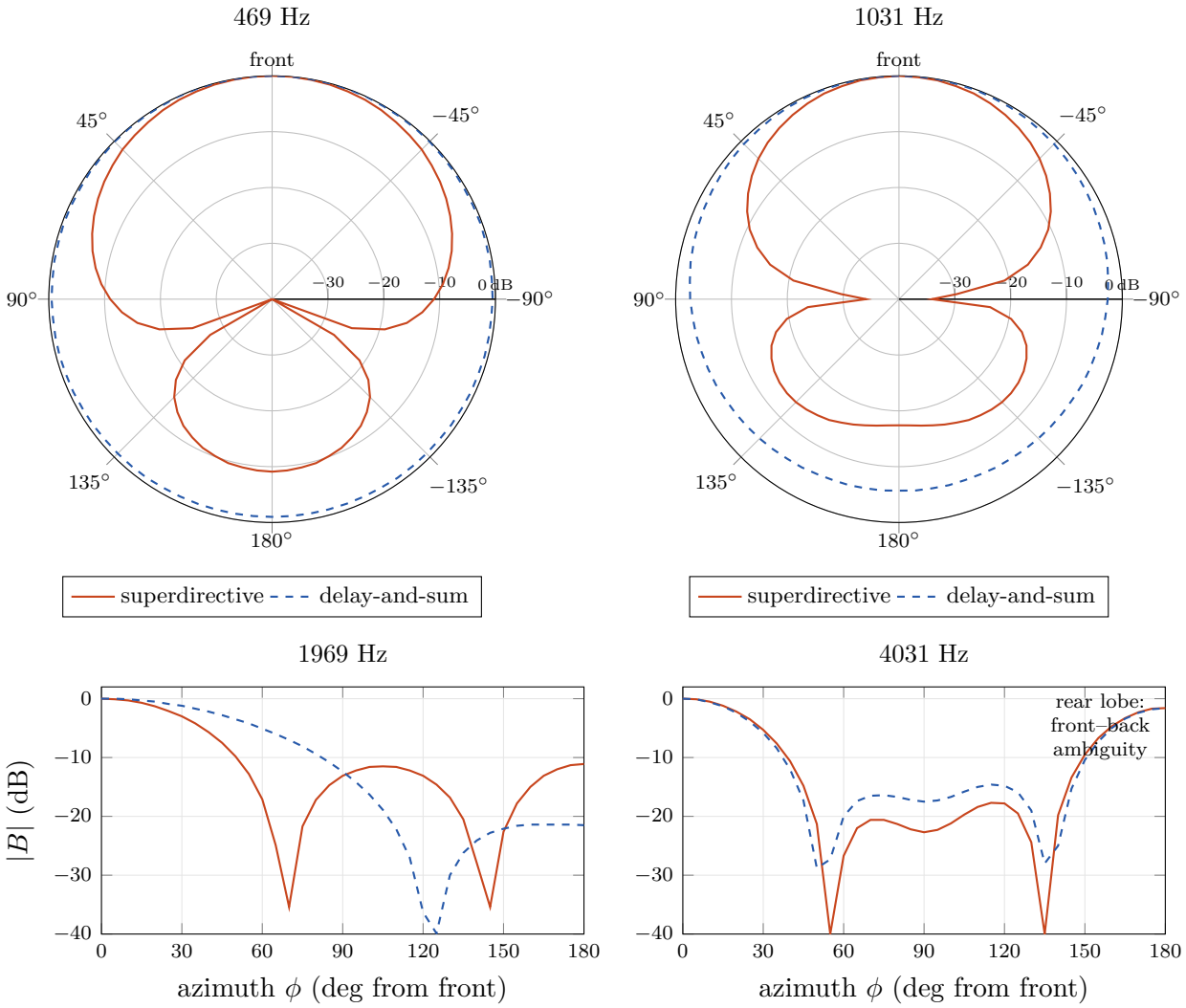


Figure 5: Beam patterns computed from `design_weights` (5° grid, values floored at −40 dB). Solid: superdirective ($\mu = 0.05$); dashed: delay-and-sum. In the polar plots the front is at the top and the left at the left. The bottom plots show only 0–180°, which is enough by Proposition 2.2.

Reading the patterns. Half-width at -3 dB, read off the 5° grid:

	469 Hz	1031 Hz	1969 Hz	4031 Hz
superdirective	$\approx 53°$	$\approx 37°$	$\approx 30°$	$\approx 23°$
delay-and-sum	none (min -1.0 dB)	$\approx 97°$	$\approx 46°$	$\approx 22°$

Below 2 kHz the superdirective design gives a genuine forward lobe where delay-and-sum is nearly omnidirectional. Above 4 kHz the two are essentially the same, as predicted in Section 5.3.

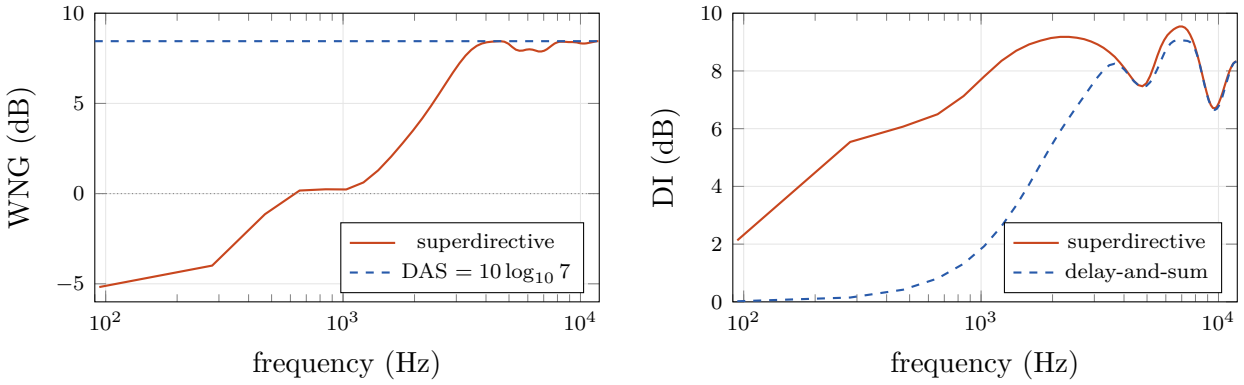


Figure 6: The core trade-off, computed for every other bin. Left: white-noise gain. Below about 600 Hz the superdirective beam *amplifies* self-noise relative to one mic (WNG < 0 dB); this is the hiss that `DIAGONAL_LOADING` controls. Right: directivity index against diffuse noise (computed with the unloaded $\mathbf{\Gamma}$). At 281 Hz the superdirective design has 5.5 dB, where DAS has 0.15 dB.

6.1 Spatial aliasing and the front–back lobe

A uniformly sampled line array obeys a *spatial* Nyquist rule: the spacing must be below $\lambda/2$, i.e. $f < c/(2d_{\min}) = 343/(2 \cdot 0.043) \approx 3.99$ kHz. A hexagon is not a line, but the same mechanism gives one clean, exact result.

Proposition 6.2 (Front/back ambiguity at 4.61 kHz)**.** *At $f^\star = c/(2r\cos 30°) \approx 4.61$ kHz, the steering vectors for $\phi = 0°$ and $\phi = 180°$ are identical. Every beamformer then has $|B(f^\star, 180°)| = |B(f^\star, 0°)| = 1$.*

Proof. For $\phi = 0°$ and $180°$, $\mathbf{u} = \pm\mathbf{e}_x$, so $d_m = e^{\pm j\kappa x_m}$, where x_m is mic m's front coordinate. The two vectors agree iff $e^{2j\kappa x_m} = 1$ for all m. The x_m take the values 0 (mics 0, 1, 4) and $\pm r\cos 30° = \pm 37.2$ mm (mics 5, 6, 2, 3). So the condition is $2\kappa \cdot 37.2\,\text{mm} \in 2\pi\mathbb{Z}$, whose smallest positive solution is $f^\star = c/(2 \cdot 0.0372)$. □

The 4031 Hz panel of Figure 5 is already approaching $f^\star$, and shows a -1.6 dB lobe straight behind the array for *both* designs. No choice of weights can fix this, because it is a property of the geometry. In practice the array gives useful rejection from about 300 Hz to 4 kHz, which covers most of the energy and intelligibility of speech.

7 Streaming: STFT analysis, beamforming, overlap-add synthesis

```
85 class StreamingBeamformer:
86     """Overlap-add STFT beamformer: 7 mic channels in, 1 beam out."""
87
88     def __init__(self, weights, hop):
89         self.hop = hop
90         self.n_fft = 2 * hop
91         self.window = np.sqrt(np.hanning(self.n_fft + 1)[:-1]).astype(np.float32)
92         self.w_h = weights.conj().astype(np.complex64)
93         self.in_buf = np.zeros((self.n_fft, weights.shape[1]), np.float32)
94         self.out_buf = np.zeros(self.n_fft, np.float32)
95
96     def process(self, block):
97         h = self.hop
98         self.in_buf[:-h] = self.in_buf[h:]
99         self.in_buf[-h:] = block
100
101        spec = np.fft.rfft(self.in_buf * self.window[:, None], axis=0)
102        beam = np.einsum("fm,fm->f", self.w_h, spec)
103        frame = np.fft.irfft(beam, n=self.n_fft).astype(np.float32) * self.window
104
105        self.out_buf += frame
106        out = self.out_buf[:h].copy()
107        self.out_buf[:-h] = self.out_buf[h:]
108        self.out_buf[-h:] = 0.0
109        return out
```

7.1 What one call to `process` computes

Index blocks so that block t holds samples $[tH, (t+1)H)$. When block t arrives, `in_buf` holds frame t, the $N = 2H$ samples $x_m[(t-1)H + n]$ for $n = 0, \dots, N-1$ (lines 98–99). Let $g[n]$ be the window. The algorithm is:

$$X_m^{(t)}[k] = \sum_{n=0}^{N-1} g[n]\, x_m[(t-1)H + n]\, e^{-j2\pi kn/N} \qquad \text{(analysis, line 101)} \tag{12}$$

$$Y^{(t)}[k] = \sum_{m} \overline{w_{k,m}}\, X_m^{(t)}[k] = \mathbf{w}_k^{\mathsf{H}} \mathbf{X}^{(t)}[k] \qquad \text{(beamform, line 102)} \tag{13}$$

$$y^{(t)}[n] = g[n] \cdot \frac{1}{N} \sum_{k=0}^{N-1} Y^{(t)}[k]\, e^{j2\pi kn/N} \qquad \text{(synthesis, line 103)} \tag{14}$$

$$y[(t-1)H + n] \mathrel{+}= y^{(t)}[n] \qquad \text{(overlap-add, line 105)} \tag{15}$$

In the synthesis line, `irfft` fills in the bins $k > N/2$ by Hermitian symmetry, $Y[N-k] = \overline{Y[k]}$, which guarantees a real output. Equivalently, the weights define real-valued filters.

The `einsum("fm,fm->f", ...)` on line 102 multiplies the two 257×7 arrays element-wise and sums over the mic index: 257 independent 7-term dot products. It never forms a 257×257 matrix, which `w_h @ spec` would.

7.2 The window and perfect reconstruction

Line 91 builds $g[n] = \sqrt{h[n]}$ from the *periodic* Hann window:

$$h[n] = \tfrac{1}{2} - \tfrac{1}{2}\cos\frac{2\pi n}{N} = \sin^2\frac{\pi n}{N}, \qquad g[n] = \sin\frac{\pi n}{N}, \qquad n = 0, \dots, N-1. \tag{16}$$

`np.hanning(N+1)[:-1]` produces exactly this: NumPy's $(N+1)$-point symmetric window has denominator N, and dropping the last sample makes it periodic. The square root of a Hann window is the *sine window*.

Proposition 7.1 (COLA, and perfect reconstruction with delay H)**.** *(a)* $h[n] + h[n+H] = 1$ *for* $0 \le n < H$*. (b) If the beamforming step were replaced by the identity (*$Y = X$*, one channel), the output of* `process` *for block* t *would be exactly input block* $t-1$*:* $y[n] = x[n-H]$*.*

Proof. (a) $h[n + N/2] = \sin^2(\pi n/N + \pi/2) = \cos^2(\pi n/N)$, so the sum is $\sin^2 + \cos^2 = 1$.

(b) With $Y = X$, the IFFT returns the windowed frame, and the synthesis window brings the total weighting to $g^2 = h$. A given sample $x[s]$ lies in exactly two frames, at offsets n and $n+H$, so the overlap-add output at s is $(h[n] + h[n+H])\, x[s] = x[s]$ by (a). For timing, see Figure 8. After frame t is added, the first H entries of `out_buf` correspond to times $[(t-1)H, tH)$, which is block $t-1$. Those entries now hold contributions from both frames covering them ($t-1$ and t), so they are complete. Line 106 emits them, and lines 107–108 shift the buffer and zero the tail for the next call. □

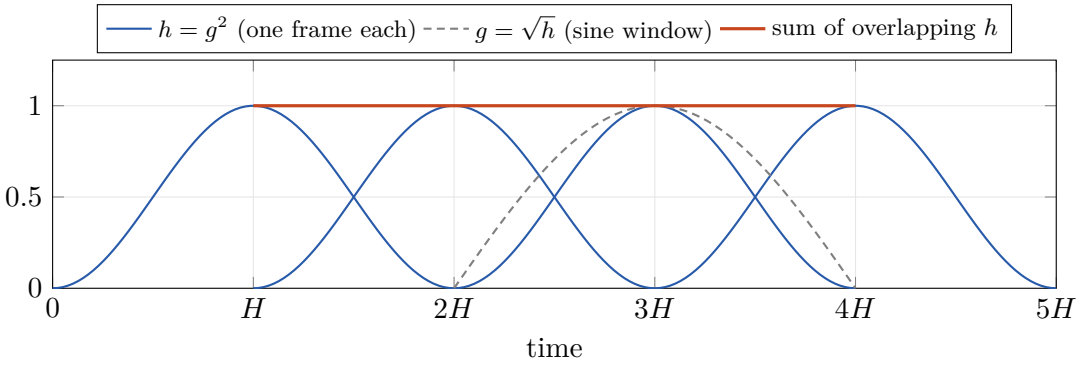


Figure 7: 50%-overlapped periodic Hann windows sum to exactly one (Proposition 7.1). The time axis is in hops; each frame spans $N = 2H$ samples.

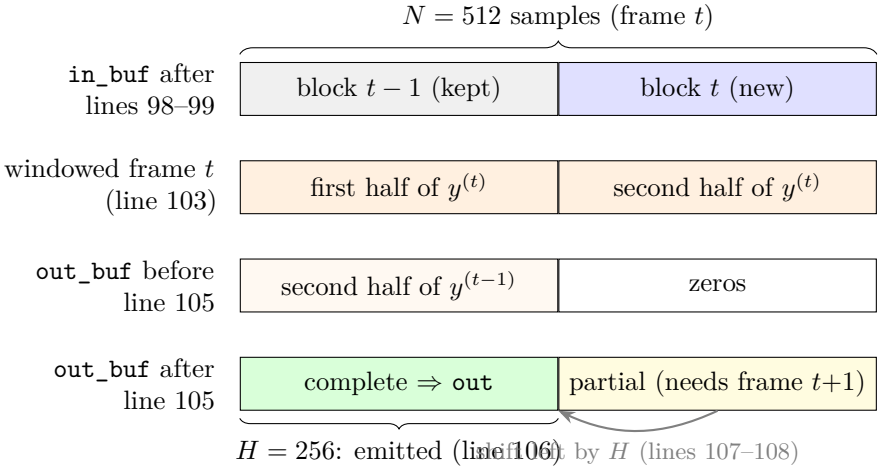


Figure 8: Buffer bookkeeping in one call to `process`. Output block t is input block $t-1$, filtered, so the algorithm adds exactly H samples of delay.

7.3 Circular convolution and how much it matters

Multiplying DFT bins is *circular* convolution. Each mic is convolved with the length-N impulse response $\mathrm{h}_m = \texttt{irfft}(\overline{w_{\cdot,m}})$, and any part of that response that extends past the frame edge wraps around to the other end. For DAS the responses are fractional delays of at most 6 samples, so the wrap is negligible. The superdirective responses are longer. Measured from `design_weights` over all seven mics, 91% of their energy lies within ± 32 samples of $n = 0$ and 99.8% within $\pm 128 = \pm N/4$. The synthesis half of the sine window tapers both frame edges to zero, which further attenuates whatever wraps. So the residual time-aliasing is small but not zero. Eliminating it completely would require constraining the filter length or zero-padding the FFT ($N_{\text{fft}} > N$), at some cost in latency and CPU.

Remark 7.2. `irfft` ignores the imaginary parts of the DC and Nyquist (24 kHz) bins, because a real signal cannot have them. The weights are real at DC (Section 5.3). At 24 kHz they are not exactly real, but that bin is far above anything the MEMS mics or the ear contribute.

Design rationales for the STFT

- **$\sqrt{\text{Hann}}$ on both sides.** The analysis taper limits spectral leakage between bins. The synthesis taper makes each processed frame fade in and out, so changes in the frame's content never produce edge discontinuities (clicks), and it suppresses circular wrap. COLA only constrains the *product* $g_a g_s$, so the symmetric split $g_a = g_s = \sqrt{h}$ gives both benefits.
- **Periodic, not symmetric, Hann.** `np.hanning(N)` uses denominator $N - 1$, and then $h[n] + h[n + H] \neq 1$, which leaves a small ripple at the hop rate. The `N+1/[:-1]` trick on line 91 makes the identity exact.
- **50% overlap ($N = 2H$).** This is the least overlap for which the sine-window pair satisfies COLA. It costs one FFT per channel per hop, the minimum possible.
- **$H = 256$ at 48 kHz.** This gives an algorithmic latency of $H/f_s = 5.33$ ms (plus device buffering) and a bin spacing of $f_s/N = 93.75$ Hz. Halving H would halve the latency but give only 2–3 bins below 300 Hz, exactly where superdirectivity earns its keep. Doubling H would give finer bins but 10.7 ms of latency, which becomes noticeable when you can also hear the room directly.
- **The `frames != BLOCK_SIZE` guard** (line 139) outputs silence for an odd-sized callback, because the overlap-add bookkeeping assumes a fixed hop.
- **Implementation details.** Conjugating the weights once (line 92) moves that work out of the callback. `float32/complex64` halves memory traffic and matches the stream's dtype; float32's 24-bit mantissa already exceeds a 24-bit converter's resolution. `.copy()` on line 106 is required: `out_buf[:h]` is a view, and line 107 overwrites it in place.

8 Gain staging, output and level metering

```
135 def audio_callback(indata, outdata, frames, time_info, status):
136     if status:
137         statuses.append(status)
138 
139     if frames != BLOCK_SIZE:
```

```
140         outdata.fill(0)
141         return
142
143     # Ripple DSP: beamform the 7 mic channels (spare channel dropped).
144     mics = indata[:, beamformer.channels]
145     beam = beamformer.process(mics) * beamformer.gain
146     np.clip(beam, -1.0, 1.0, out=beam)
147     outdata[:] = beam[:, None]        # same mono beam to both headphone channels
```

```
200     # Multiplying a float by 2**n is exact: the float equivalent of a left shift by n bits.
201     beamformer.gain = np.float32(OUTPUT_GAIN * 2.0 ** gain_shift)
```

Because the beamformer is distortionless ($\mathbf{w}^{\mathsf{H}}\mathbf{d} = 1$, Section 4), the target passes through it at unity gain. All of the loudness therefore comes from one scalar,

$$G = \texttt{OUTPUT_GAIN} \cdot 2^{s} = 8 \cdot 2^{1} = 16, \qquad 20\log_{10} 16 = 24.08 \text{ dB}, \tag{17}$$

where s is `-gain-shift`, and each step of s adds $20\log_{10} 2 = 6.02\,\text{dB}$. In IEEE-754 floating point, multiplying by 2^{s} changes only the exponent field, so it is exact: the floating-point analogue of a left shift by s bits. Since $8 = 2^{3}$ as well, G is a power of two and the scaling adds no rounding error at all.

Design rationales for the output stage

- **Why so much gain.** The RAW firmware (`micArray_vf_raw`) applies no digital gain, and MEMS mics at conversational distances sit far below full scale.
- **Why clip** (line 146). Values outside $[-1, 1]$ would be wrapped or otherwise mangled by the output path. A hard clip limits the damage to brief distortion during overloads. It does nothing at normal levels.
- **Why mono to both ears** (line 147). A beamformer produces one signal. Feeding it to both ears centres the voice in the head; the array's spatial cues are not preserved.

Level metering (`-levels`). Lines 157–158 print, for each channel, the level in dB relative to full scale over a 100 ms block: $L = 20\log_{10}(\sqrt{\frac{1}{n}\sum x^2})$ dBFS. A full-scale sine reads -3.01 dBFS, a silent channel reads about -240 dBFS (from the 10^{-12} floor), and tapping a mic makes its channel jump by tens of dB. This is how you verify the channel mapping assumed on lines 20–27.

A Parameters and their effects

Constant (line)	Value	Meaning; effect of changing it
`SAMPLE_RATE` (6)	48000	f_s. Sets bin spacing f_s/N and latency H/f_s.
`IN_CHANNELS` (7)	8	7 mics + spare, as delivered by the RAW firmware.
`OUT_CHANNELS` (8)	2	Headphones; both get the same mono beam.
`BLOCK_SIZE` (9)	256	Hop H; FFT is $2H$. Larger: finer bins, more latency.
`RING_MIC_ANGLES_DEG` (20)	see Fig. 2	Mic angles θ_m. Must match the hardware, or the beam mis-steers.
`RING_RADIUS` (29)	0.043 m	r. Sets all spacings, the aliasing limit and low-frequency directivity.
`LOOK_AZIMUTH_DEG` (31)	0	ϕ_0. Steers the beam; any angle works because $\mathbf{d}$ is computed, not tabulated.
`SPEED_OF_SOUND` (32)	343 m/s	c at about 20°C. A few-% error slightly mis-scales τ_m.
`DIAGONAL_LOADING` (33)	0.05	μ. Higher: less hiss, wider beam (tends to DAS). Lower: narrower beam, more hiss.
`OUTPUT_GAIN` (34)	8	Fixed post-beam gain (18.1 dB).
`GAIN_SHIFT_BITS` (35)	1	Default `-gain-shift`; +6.02 dB per step.

B Symbols

M	number of mics (7)	$\mathbf{p}_m$	position of mic m
f_s	sample rate	$\mathbf{u}$	unit vector toward source
$H,\ N$	hop (256), FFT length (512)	τ_m	arrival advance at mic m
$k,\ f_k$	bin index, kf_s/N	κ	wavenumber $2\pi f/c$
$\phi,\ \phi_0$	azimuth, look azimuth	$\mathbf{d}(f,\phi)$	steering vector
$\mathbf{w}(f)$	beamformer weights	$\mathbf{\Gamma},\ \mathbf{\Gamma}_\mu$	diffuse coherence, loaded
μ	diagonal loading	$B(f,\phi)$	beam pattern $\mathbf{w}^{\mathsf{H}}\mathbf{d}$
WNG	white-noise gain $1/\lVert\mathbf{w}\rVert^2$	DI	diffuse array gain
$g,\ h$	sine window, periodic Hann	G	output gain $8 \cdot 2^s$